Hotel Management System – Reverse Engineering & Refactoring Report

Course Task Submission (NetBeans UML + Code Review & Refactoring)

Student Name: Abd-AL-Jwabrah

Student ID: 201810363

Date:1/9/2026

GitHub Link: https://github.com/jwabrah2/Hotel-Management-UML-Refactor

---

1. Introduction

This report documents the work completed for reverse engineering and refactoring the provided Java Hotel Management System project. The tasks included importing the project into NetBeans, generating a UML class diagram from source code using a UML plugin, reviewing the project code, and applying refactoring improvements. Finally, the updated code and UML artifacts are prepared for pushing to GitHub.

2. Task Requirements

The assignment required the following tasks:

• Task 1: Reverse engineer the provided Java project using NetBeans and a UML plugin to generate a UML class diagram from source code.

• Task 2: Review the source code and apply refactoring to improve quality, correctness, and maintainability.

• Task 3: Push the generated UML and refactored code to GitHub and submit the repository link via eLearning.

3. Tools and Environment

The following tools were used to complete the tasks:

• Apache NetBeans IDE (Java SE)

• UML Plugin for NetBeans (used to generate UML from source code)

• SQLite JDBC Driver (sqlite-jdbc library)

• Git and GitHub (version control and online repository)

4. Task 1 – Reverse Engineering and UML Class Diagram

The project was imported into NetBeans and the UML plugin was used to generate a UML class diagram from the existing source code. The diagram provides a structural view of the main classes such as Booking, Room, RoomFare, UserInfo, Payment, and the related database access classes (BookingDb, CustomerDb, RoomDb, ItemDb, OrderDb).

5. Task 2 – Code Review and Refactoring

A code review was performed to identify weaknesses such as SQL injection risks, duplicated code, broken SQL queries, missing resource handling, and lack of documentation. The refactoring focused on improving safety, readability, and maintainability without changing the project's functional behavior.

5.1 Refactoring Summary

• Replaced SQL string concatenation with parameterized PreparedStatements to prevent SQL injection and quote errors.

• Introduced try-with-resources in database classes to automatically close statements and reduce resource leaks.

• Fixed broken and incorrect SQL statements (notably ItemDb.updateItem and RoomDb.updateRoom).

• Added null-safety checks to prevent NullPointerException in UI/table rendering and models.

• Introduced constants and removed magic numbers for clarity (e.g., BookingTableModel displayed days/columns).

• Improved naming consistency and removed unused imports in refactored files.

• Added comprehensive Javadoc comments for all refactored classes and methods.

5.2 Detailed Refactoring by File

A) ItemDb.java (Database Layer)

Issues found:
• SQL Injection risk due to query concatenation.
• Critical bug: updateItem was updating the wrong table (food) and had invalid SQL syntax.
Refactoring applied:
• Converted queries to parameterized PreparedStatements.
• Fixed SQL to UPDATE item table correctly and updated name/description/price safely.

• Used try-with-resources for insert/update/delete.
• Added full Javadoc.

B) OrderDb.java (Database Layer)

Issues found:
• SQL Injection risk.
• Manual statement flushing.
Refactoring applied:
• Converted INSERT query to parameterized PreparedStatement.
• Used try-with-resources.
• Added null checks and documentation.

C) RoomDb.java (Database Layer)

Issues found:
• SQL Injection risk.
• Broken updateRoom query: missing WHERE clause and contained an incomplete field assignment (meal_id =).
• Repeated DB resource management using flush methods.
Refactoring applied:
• Converted INSERT/UPDATE/DELETE operations to parameterized PreparedStatements.
• Fixed updateRoom query by adding WHERE room_id = ? and removing the broken incomplete SQL part.
• Added constants and safer boolean handling.
• Added Javadoc.

D) BookingTableModel.java (TableModel Layer)

Issues found:
• Potential NullPointerException when data is not initialized.
• Month day calculation did not support leap years.
• Repeated object creation and inefficient date formatting.
Refactoring applied:
• Added null-safety checks and bounds checks for table access.
• Added constants for the number of days/columns.
• Added leap-year support for February.
• Reduced repeated database/helper object creation.
• Added fireTableDataChanged() to update UI.
• Added Javadoc.

E) CustomCellRenderer.java (UI Table Rendering)

Issues found:
• Potential NullPointerException when cell value is null.
• Limited marker coloring and redundant background override.

Refactoring applied:
• Added null-safe value-to-string conversion.
• Preserved selection rendering.
• Improved marker coloring for symbols (>, <, <<, >>).
• Removed unused imports and added Javadoc.

F) ControlPanel.java (UI Layer)

Issues found:
• Mixed UI logic with database logic and duplicated validations.
• Raw collection types without generics.
• Some ResultSets not closed safely.
Refactoring applied (minimal change to preserve NetBeans GUI Builder):
• Introduced generics and renamed fields for clarity.
• Added helper methods (isBlank, digitsOnly) to reduce duplication.
• Refactored event handlers for numeric input validation.
• Improved room selection logic using String.join.
• Improved customer search handling and safer ResultSet usage.
• Added method-level documentation.

6. Task 3 – GitHub Submission

After generating the UML diagram and completing refactoring, the updated project files were pushed to GitHub. The repository contains:
• The refactored Java source code.
• UML class diagram (image or exported UML project).
• This report (optional).

GitHub Repository Link: https://github.com/jwabrah2/Hotel-Management-UML-Refactor

7. Conclusion

The project was successfully reverse engineered and refactored. The UML class diagram provides a clear view of the system structure, and the applied refactoring improved code safety, correctness, and maintainability (especially in database access code). The final deliverables were prepared for GitHub submission as required.

Appendix A – Screenshots / Evidence

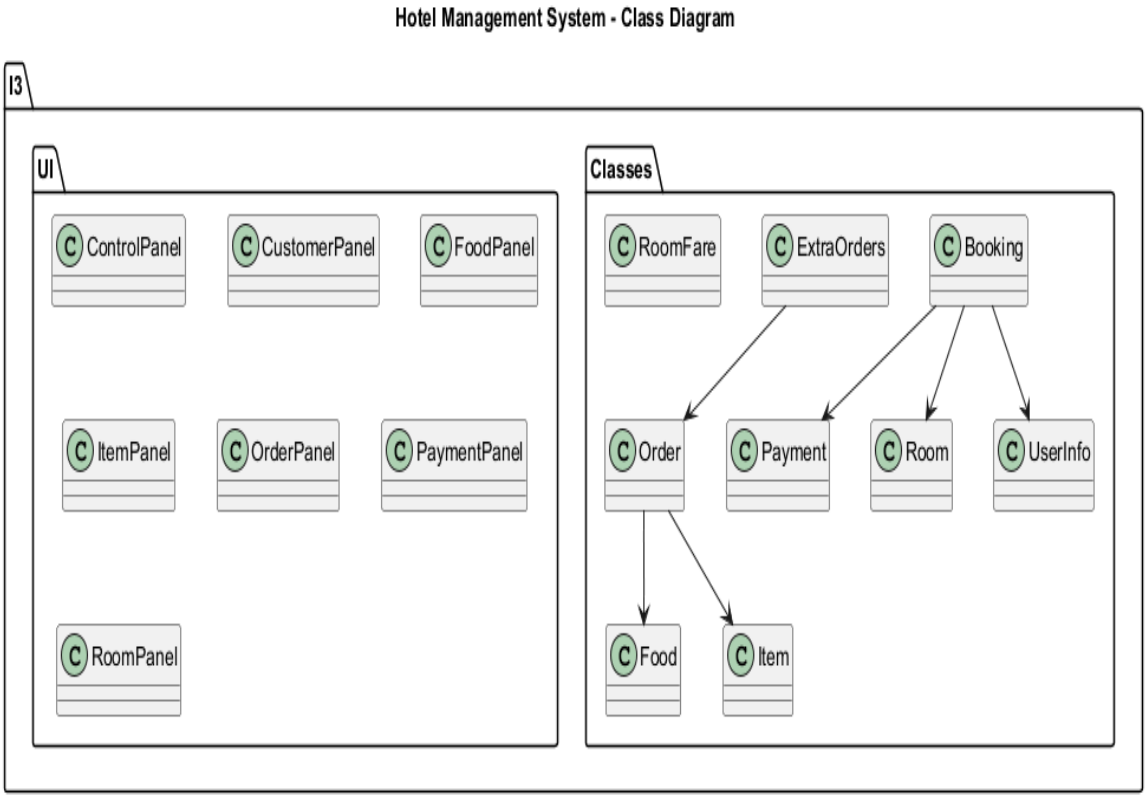

Figure 1 – UML Class Diagram

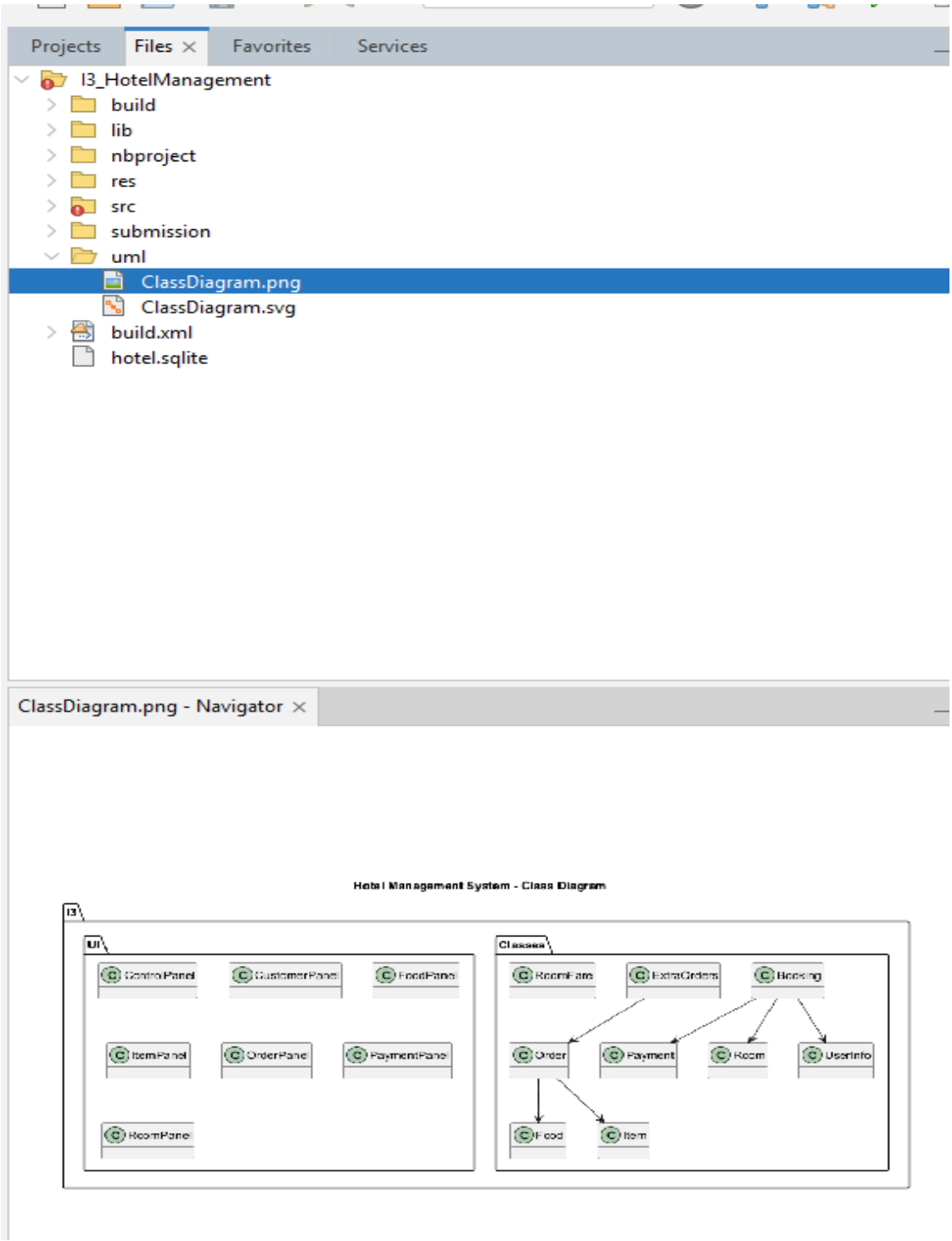

Figure 2 – NetBeans UML generation